

제 3장 제어 메시지 처리하기

2026년 1학기 윈도우 프로그래밍

학습 목표

- 학습 목표

- 마우스에서 발생한 메시지를 이용할 수 있다.
- 타이머를 이용해 자동으로 움직이는 형상을 윈도우에 표현할 수 있다.

- 내용

- 마우스 메시지 처리하기
- 타이머 메시지 처리하기
- 래스터 연산

1. 마우스 메시지

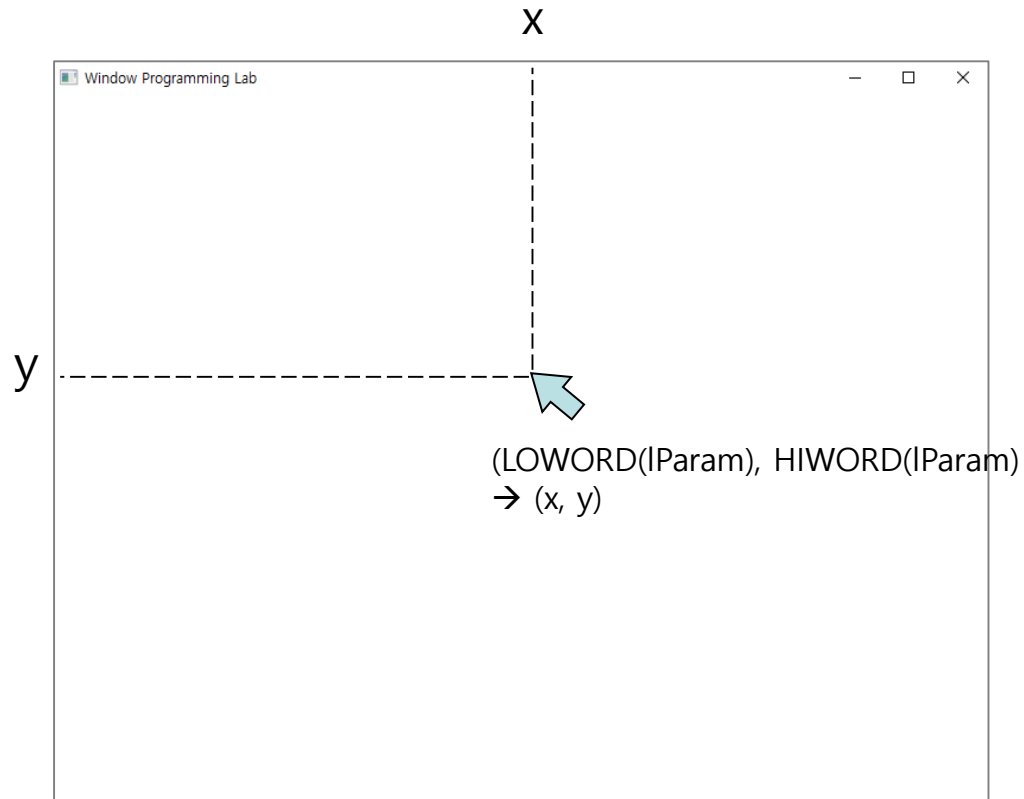
- **마우스 이벤트**
 - 윈도우와의 상호작용에서 가장 빈번하게 발생하는 이벤트
 - 클릭, 이동 (드래그), 더블클릭 등의 메시지로 표현됨

- **좌우 버튼이 있는 마우스의 이벤트**

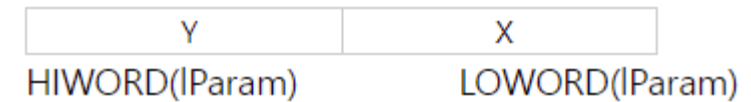
메시지	내용	윈도우 프로시저 인수값
WM_LBUTTONDOWN	왼쪽 마우스 버튼을 눌렀을 때 발생하는 메시지	wParam: 키보드와 다른 마우스 버튼의 현재 상태를 알린다. MK_CONTROL: ctrl 키가 눌러져 있다. MK_LBUTTON: 마우스 왼쪽 버튼이 눌러져 있다. MK_RBUTTON: 마우스 오른쪽 버튼이 눌러져 있다. MK_MBUTTON: 마우스 중간 버튼이 눌러져 있다. MK_SHIFT: shift 키가 눌러져 있다. lParam: x와 y 좌표값이 저장 HIWORD (lParam): y 값 LOWORD (lParam): x 값
WM_LBUTTONUP	왼쪽 마우스 버튼을 떼었을 때 발생하는 메시지	
WM_RBUTTONDOWN	오른쪽 마우스 버튼을 눌렀을 때 발생하는 메시지	
WM_RBUTTONUP	오른쪽 마우스 버튼을 떼었을 때 발생하는 메시지	
WM_MOUSEMOVE	마우스를 움직일 때 발생하는 메시지	
WM_LBUTTONDOWNBLCLK / WM_RBUTTONDOWNBLCLK	버튼 더블 클릭 눌렀을 때 발생하는 메시지 윈도우 클래스가 반드시 CS_DBLCLKS 스타일을 가져야 한다	

마우스 좌표 구하기

- 마우스에 대한 데이터 값은 lParam 에 저장
 - `int y = HIWORD (lParam)`
 - `int x = LOWORD (lParam)`



- **HIWORD**: 32bit 데이터에서 상위 16bit 데이터를 구하기 위한 매크로 함수
- **LOWORD**: 32bit 데이터에서 하위 16bit 데이터를 구하기 위한 매크로 함수



예 1) 마우스로 원 선택하기

- (50, 50) 위치에 반지름이 40인 원을 그리고, 마우스를 원 내부에 클릭하면 테두리에 사각형을 그린다.

```
#include <math.h>
```

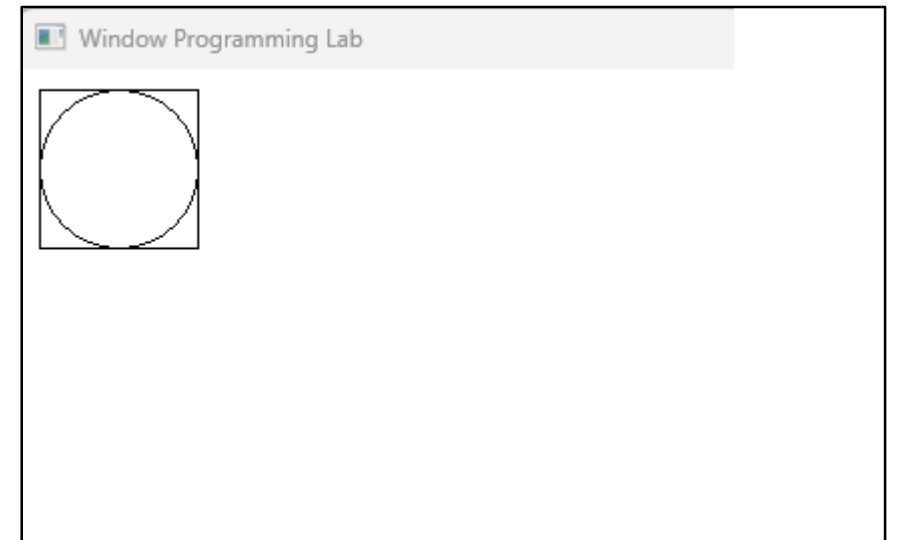
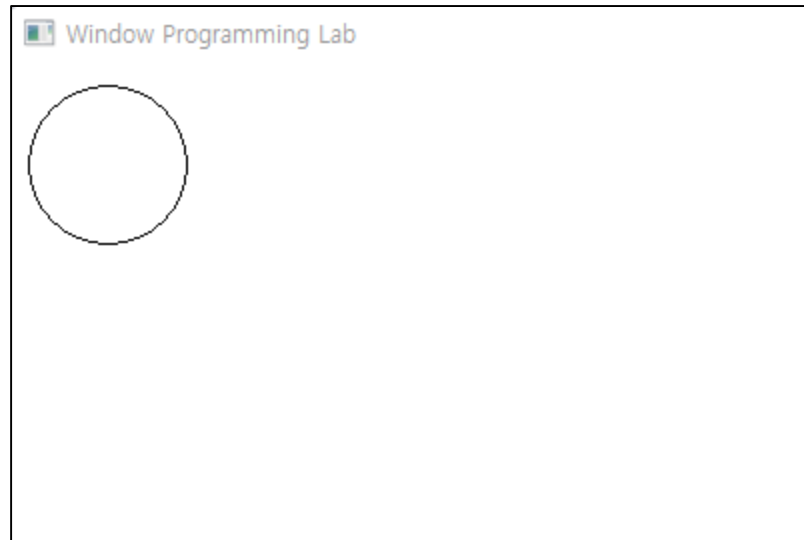
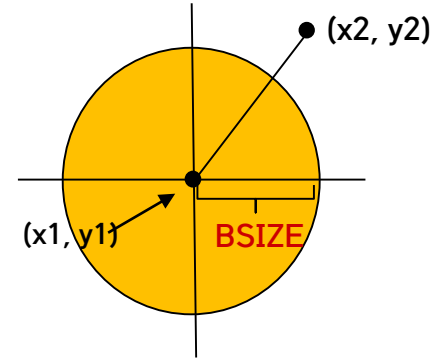
```
#define BSIZE 40 //--- 반지름
```

```
//--- (x1, y1)과 (x2, y2)간의 길이
```

```
float GetDistance (int x1, int y1, int x2, int y2)
{
    return (sqrt((x2-x1)*(x2-x1) +(y2-y1)*(y2-y1)));
}
```

```
//--- (x1, y1)과 (x2, y2)의 길이가 반지름보다 짧으면 true, 아니면 false
```

```
BOOL InCircle (int x1, int y1, int x2, int y2)
{
    if (GetDistance (x1, y1, x2, y2) < BSIZE) //--- BSIZE: 반지름
        return TRUE;
    else
        return FALSE;
}
```



예 1) 마우스로 원 선택하기(계속)

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    HDC hDC;
    static int centerX, centerY, mx, my;
    static BOOL Selection;

    switch (iMsg)
    {
        case WM_CREATE :
            centerX = 50;           centerY = 50;           //--- 원의 중심점
            Selection = FALSE;       //--- 원이 선택되었나, FALSE : 아직 안되었음
            break;

        case WM_PAINT :
            hDC = BeginPaint (hWnd, &ps) ;
            if (Selection)           //--- 만약 원이 선택되었다면, 사각형을 그린다.
                Rectangle(hDC, centerX -BSIZE, centerY -BSIZE, centerX +BSIZE, centerY +BSIZE);    //--- 테두리 사각형을 그린다.
                Ellipse (hDC, centerX -BSIZE, centerY -BSIZE, centerX +BSIZE, centerY +BSIZE);        //--- 원을 그린다.
                EndPaint (hWnd, &ps) ;
            break;

        case WM_LBUTTONDOWN :           //--- 왼쪽 버튼 누르면
            mx = LOWORD (lParam);         //--- 마우스 좌표값 저장: (mx, my)
            my = HIWORD (lParam);
            if (InCircle (centerX, centerY, mx, my))
                Selection = TRUE;         //--- 원의 중심점(centerX, centerY)와 마우스 좌표 (mx, my) 비교
                InvalidateRect (hWnd, NULL, TRUE);    //--- 원 안에 있으면 true
            break;

        case WM_LBUTTONUP :           //--- 왼쪽 버튼을 놓으면
            Selection = FALSE;         //--- selection은 false
            InvalidateRect (hWnd, NULL, TRUE);
            break;

        case WM_DESTROY:
            PostQuitMessage (0);
            break;
    }
    return (DefWindowProc (hWnd, iMsg, wParam, lParam));
}
```

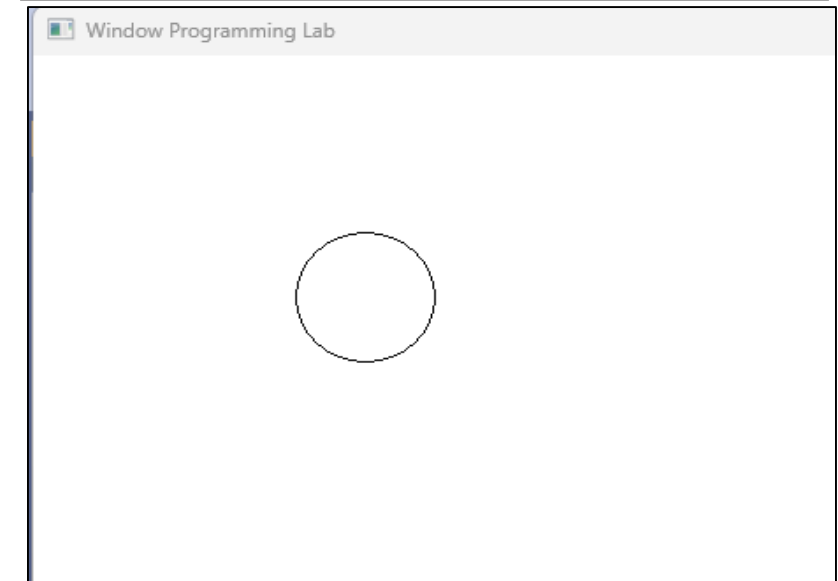
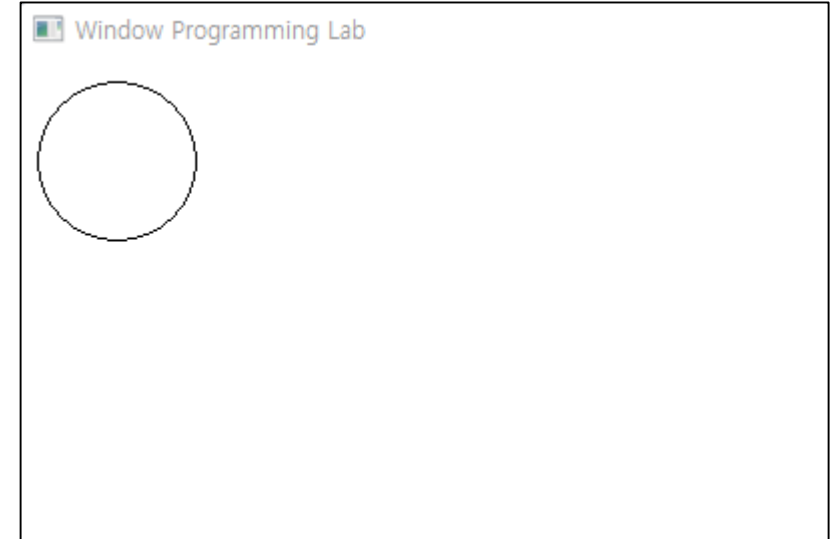
예 2) 마우스 드래그로 원 이동하기

- 왼쪽 마우스 버튼으로 원을 선택한 후 마우스를 드래그하면 원이 마우스의 위치로 이동된다.

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    static int centerX, centerY;
    static BOOL Selection;
    int mx, my;

    switch (iMsg)
    {
        //--- WM_CREATE 메시지는 앞 예제의 메시지 처리 사용
        case WM_PAINT :
            hDC = BeginPaint (hWnd, &ps) ;
            Ellipse (hDC, centerX -BSIZE, centerY -BSIZE, centerX +BSIZE, centerY +BSIZE);    //--- 원을 그린다.
            EndPaint (hWnd, &ps) ;
            break;
        case WM_LBUTTONDOWN :
            mx = LOWORD(lParam);
            my = HIWORD(lParam);
            if (InCircle (centerX, centerY, mx, my))
                Selection = TRUE;                                //--- mx, my : 마우스 좌표
            InvalidateRect (hWnd, NULL, TRUE);
            break;
        case WM_LBUTTONUP :
            Selection = FALSE;
            InvalidateRect (hWnd, NULL, TRUE);
            break;
        case WM_MOUSEMOVE:
            mx = LOWORD(lParam);
            my = HIWORD(lParam);
            if (Selection)                                     //--- 원이 선택된 상태로 움직이면
            {
                centerX = mx;    centerY = my;
                InvalidateRect (hWnd, NULL, TRUE);            //--- 원 그리기
            }
            break;
    }
    return (DefWindowProc (hWnd, iMsg, wParam, lParam));
}
```



예 3) 더블 클릭

- 왼쪽 마우스 버튼 더블 클릭하면 도형을 그리거나 지우거나 한다.

```
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdParam, int nCmdShow)
{
    HWND hWnd;
    MSG Message;
    WNDCLASSEX WndClass;
    g_hInst = hInstance;

    WndClass.cbSize = sizeof(WndClass);
    WndClass.style = CS_HREDRAW | CS_VREDRAW | CS_DBLCLKS;           //--- 더블 클릭을 사용할 수 있게 스타일 설정한다.
    WndClass.lpfnWndProc = (WNDPROC)WndProc;
    WndClass.cbClsExtra = 0;
    WndClass.cbWndExtra = 0;
    WndClass.hInstance = hInstance;
    WndClass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
    WndClass.hCursor = LoadCursor(NULL, IDC_ARROW);
    WndClass.hbrBackground = (HBRUSH)GetStockObject(WHITE_BRUSH);
    WndClass.lpszMenuName = NULL;
    WndClass.lpszClassName = lpszClass; // lpszClass;
    WndClass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
    RegisterClassEx(&WndClass);

    ...
}
```


예 3) 더블 클릭

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMessage, WPARAM wParam, LPARAM lParam)

```
{
    HDC hDC;
    PAINTSTRUCT ps;
    HPEN hPen, oldPen;
    static int draw = 1;

    switch (iMessage) {
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);                //--- DC 얻어오기

            if (draw) {                                     //--- 더블클릭을 하면 번갈아가며 원을 그린다/안그린다.
                hPen = CreatePen (PS_DOT, 1, RGB(255, 0, 0)); //--- GDI: 펜 만들기
                oldPen = (HPEN)SelectObject (hDC, hPen);      //--- 새로운 펜 선택하기

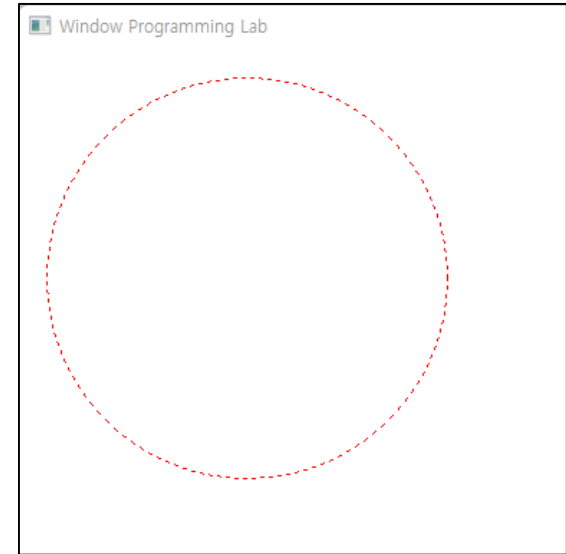
                Ellipse (hDC, 20, 20, 300, 300);           //--- 선택한 펜으로 도형 그리기

                SelectObject (hDC, oldPen);                 //--- 이전의 펜으로 돌아감
                DeleteObject(hPen);                        //--- 만든 펜 객체 삭제하기
            }

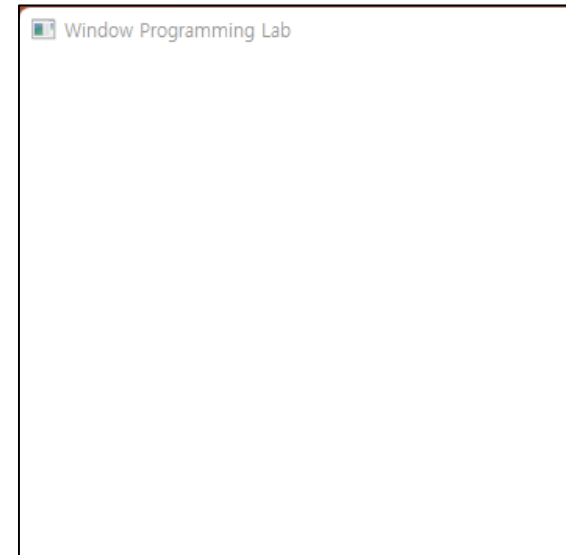
            EndPaint (hWnd, &ps);                          //--- DC 해제하기
            break;

        case WM_LBUTTONDOWNBLCLK:
            draw++;
            draw %= 2;
            InvalidateRect (hWnd, NULL, TRUE);
            break;

        case WM_DESTROY:
            PostQuitMessage (0);
            break;
    }
    return (DefWindowProc (hWnd, iMessage, wParam, lParam));
}
```



클릭 → 그리기
다시 클릭 → 지우기



2. 타이머 메시지

- 타이머 메시지
 - 응용 프로그램에 키보드나 마우스 버튼의 입력이 아니라 다른 신호를 주기적으로 주고 싶을 때
 - 발생하는 메시지: **WM_TIMER**
- 특정 함수로 타이머를 설치했을 경우, 지정한 시간 간격으로 **WM_TIMER** 메시지가 반복적으로 큐에 넣어진다.
 - WM_TIMER 메시지 발생 시:
 - **wParam: 타이머의 ID**
 - **lParam: 타이머 콜백 함수 이름 (콜백함수를 사용하지 않으면 NULL)**
 - 한 개 이상의 타이머를 설정할 수 있다.
 - 여러 개의 타이머가 설치되어 있을 경우:
 - 각각의 타이머는 정해진 시간 간격으로 이 메시지를 큐에 저장
 - 어떤 타이머에 의해 이 메시지가 발생했는지 **wParam**값 (타이머 id)으로 조사한다.
 - 타이머 설정 함수: **SetTimer ();**

2. 타이머 메시지

- WM_TIMER 메시지는 다른 메시지들에 비해 **우선순위가 낮게 설정**
 - 먼저 처리해야 할 메시지가 있을 경우 곧바로 윈도우 프로시저로 보내지지 않을 수 있다.
 - 따라서 정확한 시간에 이 메시지가 전달되지 않는 경우도 있으므로 정확도를 요하는 작업에는 이 메시지를 사용하지 않는 것이 좋다.
 - 정확도를 요하는 작업에는 **타이머 콜백 함수를 지정**한다.
 - 타이머 콜백 함수를 지정했을 경우는 매번 이 메시지를 수행하는 것이 아니라, 프로그래머가 만든 함수(타이머 콜백 함수)를 **OS가 자동으로 주기적으로 호출**해 준다.
- 따라서 타이머를 사용할 때는
 - 1) 타이머 메시지를 사용 하거나
 - 2) 타이머 콜백 함수를 사용한다.

타이머 메시지

- 타이머 설정함수
 - 타이머를 설정/해제하는 함수

WORD SetTimer (HWND hWnd, UINT_PTR nIDEvent, UINT uElapse, TIMERPROC lpTimerFunc);

- hWnd : 윈도우 핸들
- nIDEvent : 타이머 ID, 여러 개의 타이머를 구분하기 위한 정수
- uElapse : 시간간격 milisec 단위(1000분의 1초)
- lpTimerFunc : 시간간격 마다 수행할 함수
 - NULL이라고 쓰면 WndProc() 함수가 타이머메시지를 처리)

BOOL KillTimer (HWND hWnd, UINT_PTR uIDEvent);

- hWnd: 윈도우 핸들
- uIDEvent: 삭제할 타이머의 ID

- 타이머 콜백 함수
 - SetTimer 함수의 마지막 인자로 설정되는 타이머 콜백 함수

void CALLBACK TimerProc (HWND hWnd, UINT uMsg, UINT_PTR idEvent, DWORD dwTime);

- hWnd: 타이머를 소유한 윈도우 핸들
- uMsg: WM_TIMER 메시지
- idEvent: 타이머 id
- dwTime: 윈도우가 실행된 후의 경과시간

타이머 메시지

- 2개의 타이머를 설정하고, 각각 타이머에 따라 문자를 출력하고, 위치를 점점 오른쪽으로 이동한다.

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

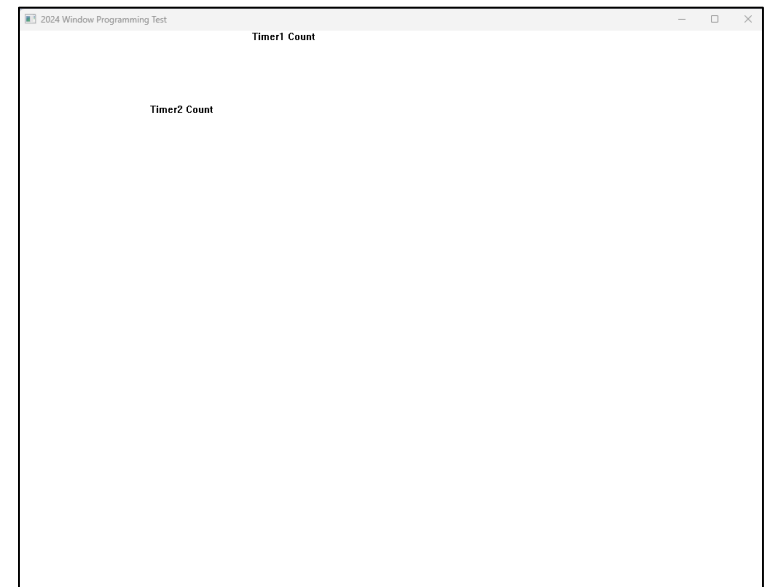
```
{
    HDC hDC;
    static int Timer1Count=0, Timer2Count=0;

    switch (iMsg) // 메시지 번호
    {
        case WM_CREATE:
            SetTimer (hWnd, 1, 60, NULL);           //--- 1번 아이디를 가진 타이머: 0.06초 간격 → 타이머 메시지 사용
            SetTimer (hWnd, 2, 100, NULL);          //--- 2번 아이디를 가진 타이머: 0.1초 간격 → 타이머 메시지 사용
            break;

        case WM_TIMER:
            switch (wParam) {
                case 1:                               //--- 1번 아이디 타이머: 0.06초 간격으로 실행
                    Timer1Count++;
                    break;
                case 2:                               //--- 2번 아이디 타이머: 0.1초 간격으로 실행
                    Timer2Count++;
                    break;
            }
            InvalidateRect (hWnd, NULL, TRUE);
            break;

        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            if ( Timer1Count % 2 == 0 )                //--- Timer1Count 가 0이면 y=0 줄에 출력
                TextOut (hDC, Timer1Count*10, 0, L"Timer1 Count", 12);
            if ( Timer2Count % 2 == 1 )                //--- Timer2Count 가 1이면 y=100 줄에 출력
                TextOut (hDC, Timer2Count*10, 100, L"Timer2 Count", 12);
            EndPaint (hWnd, &ps);
            break;
    }

    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```



타이머 메시지

- 타이머 콜백 함수 이용 방법

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    switch (iMsg ){
        case WM_CREATE:
            SetTimer (hWnd, 3, 500, (TIMERPROC) TimerProc);    //--- 3번 아이디의 타이머가 0.5초 마다 TimerProc 타이머 함수 실행 → 타이머 메시지 대신 콜백 함수 사용
            break;
    }
    return 0;
}
```

void CALLBACK **TimerProc** (HWND hWnd, UINT iMsg, UINT idEvent, DWORD dwTime) //--- 3번 아이디 타이머 함수

```
{
    HDC hDC;
    HBRUSH MyBrush, OldBrush;

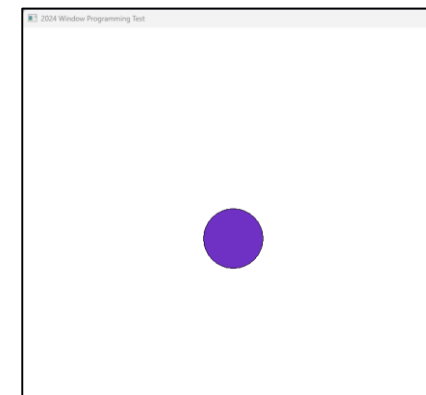
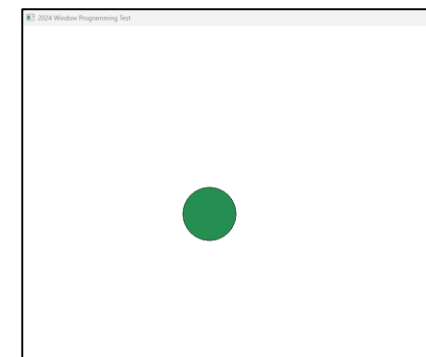
    hDC = GetDC (hWnd);
    GetClientRect (hWnd, &rect);

    MyBrush = CreateSolidBrush (RGB(rand()%255, rand()%255, rand()%255) );
    OldBrush = (HBRUSH) SelectObject (hDC, MyBrush);

    Ellipse (hDC, 300, 300, 400, 400 );    //--- (300, 300) (400, 400) 위치에 원이 그려지고 0.5초마다 색상이 변한다.

    SelectObject (hDC, OldBrush);
    DeleteObject (MyBrush);

    ReleaseDC (hWnd, hDC);
}
```



예) 타이머 메시지

- 타이머를 이용하여 시계 구현하기

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    HDC hDC;
    PAINTSTRUCT ps;
    SYSTEMTIME curTime;
    static TCHAR str[100];

    switch (iMsg ){
        case WM_CREATE:
            SetTimer (hWnd, 1, 1000, NULL);           //--- 1번 아이디의 타이머가 1초 마다 WM_TIMER 메시지 발생
            break;
        case WM_TIMER:
            GetLocalTime (&curTime);
            wsprintf (str, TEXT("current time: %d: %d: %d"), curTime.wHour, curTime.wMinute, curTime.wSecond);
            InvalidateRect (hWnd, NULL, TRUE);
            break;
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            TextOut (hDC, 0, 0, str, lstrlen(str));
            EndPaint (hWnd, &ps);
            break;
        case WM_DESTROY:
            PostQuitMessage (0);
            break;
    }
    return 0;
}
```

```
void GetLocalTime(
    [out] LPSYSTEMTIME lpSystemTime
);
```

```
typedef struct _SYSTEMTIME {
    WORD wYear;
    WORD wMonth;
    WORD wDayOfWeek;
    WORD wDay;
    WORD wHour;
    WORD wMinute;
    WORD wSecond;
    WORD wMilliseconds;
} SYSTEMTIME, *PSYSTEMTIME, *LPSYSTEMTIME;
```

 Window Programming Lab

current time : 16:7:28

예) 타이머 메시지

- 오른쪽 화살표 키를 누르면 타이머가 설정되고, 왼쪽에 있던 원이 오른쪽으로 이동한다. 원이 오른쪽 가장자리에 도착하면 반대편 가장자리로 이동해서 다시 오른쪽으로 이동한다.

LRESULT CALLBACK **WndProc** (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    HDC hDC;
    PAINTSTRUCT ps;
    static int x=0, y;
    static RECT rectView;
    switch (iMsg)
    {
        case WM_CREATE:
            GetClientRect (hWnd, &rectView);
            x = 20;    y = 20;
            break;
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            Ellipse (hDC, x-20, y-20, x+20, y+20);
            EndPaint (hWnd, &ps);
            break;
        case WM_KEYDOWN:
            if (wParam == VK_RIGHT)
                SetTimer (hWnd, 1, 70, NULL);
            break;
        case WM_TIMER:
            x += 40;
            if (x + 20 > rectView.right)
                x = 0;
            InvalidateRect (hWnd, NULL, TRUE);
            break;
        case WM_DESTROY :
            KillTimer (hWnd, 1);
            PostQuitMessage (0) ;
            break ;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```

//--- 메시지 종류

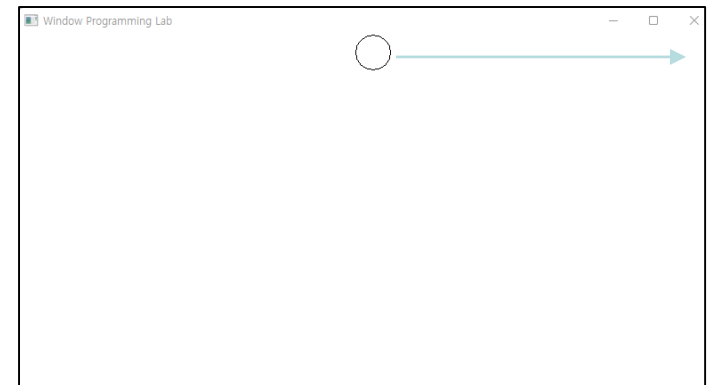
//--- 원 그리기

//--- 오른쪽 키를 누를 때
//--- 타이머 설정 → 타이머 메시지 사용

//--- 타이머 메시지: 설정된 시간마다 생성

//--- 오른쪽 가장자리에 도착하면 왼쪽 가장자리로 이동

//--- 윈도우 종료 시 타이머도 종료



3. 래스터 연산

- 래스터 연산
 - 윈도우의 배경색과 그리는 색을 연산한 결과 색상으로 그림
 - AND, OR, XOR 등 비트간의 이진 연산과 NOT 연산의 조합으로 지정됨
 - 그리기 연산은 래스터 디바이스에만 적용되며 벡터 디바이스에는 적용되지 않음
- 래스터 연산 설정 함수

int SetROP2 (HDC hDC, int fnDrawMode);

- 두 픽셀 사이에 bit 연산을 수행하도록 mix 모드를 설정할 수 있는 기능
- HDC hDC: 디바이스 컨텍스트 핸들
- int fnDrawMode: 그리기 모드 (아래 표 참고)

3. 래스터 연산

그리기 모드	의미	첫 번째 그리기	같은 위치에 다시 그리기	특징
R2_BLACK	픽셀은 항상 0(검정색)이 된다	검정색으로 그림	계속 검정 유지	항상 검정
R2_COPYPEN	픽셀은 사용된 펜의 색상으로 칠해진다 (디폴트 값)	펜 색으로 그림	계속 펜 색 유지	항상 펜 색 그대로 표시
R2_MERGEPEN	펜의 색상과 배경을 OR 시킨다	펜 색과 배경 OR	다시 그려도 같은 결과	밝은 쪽으로 합성
R2_MASKPEN	펜의 색상과 배경을 AND 시킨다	펜 색과 배경 AND	다시 그려도 같은 결과	어두운 쪽으로 합성
R2_XORPEN	펜의 색상과 배경을 XOR 시킨다	펜 색과 배경 XOR → 새로운 색	다시 XOR → 원래 배경색 복원	두 번 그리면 선이 사라짐 (토글 효과)
R2_WHITE	픽셀은 항상 1(흰색)이 된다	흰색으로 그림	계속 흰색 유지	항상 흰색
R2_NOT	배경색을 반전시킨다	배경색 반전	다시 반전 → 원래 색 복원	같은 위치에 두 번 그리면 원래 색으로 돌아옴
R2_MASKPENNOT	펜의 색상과 배경을 반전시켜 AND 연산한다.	펜 색과 배경 반전 AND	다시 그려도 같은 결과	특수 합성 효과
R2_MERGEPENNOT	배경색을 반전시켜 펜의 색상과 OR 연산한다	펜 색과 배경 반전 OR	다시 그려도 같은 결과	특수 합성 효과
R2_NOTCOPYPEN	펜의 색상을 반전시켜 칠한다	펜 색 반전	계속 반전된 색 유지	펜 색의 보색으로 표시
R2_NOTMASKPEN	R2_MASKPEN의 반전효과	펜 색과 배경 AND 후 반전	다시 그려도 같은 결과	반전된 마스크 효과
R2_NOTMERGEPEN	R2_MERGEPEN의 반전효과	펜 색과 배경 OR 후 반전	다시 그려도 같은 결과	반전된 병합 효과
R2_NOTXORPEN	R2_XORPEN의 반전효과	XOR 결과 반전	다시 그리면 원래 배경색 복원	XOR과 유사, 토글 효과
R2_NOP	픽셀은 아무런 영향을 받지 않는다	아무 것도 안 그림	계속 아무 변화 없음	선이 그려지지 않음

예 1) 래스터 연산 적용하여 선분 그리기

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hdc;
    switch (message) {
    case WM_PAINT:
        hdc = BeginPaint(hWnd, &ps);

        //--- 배경은 흰색 바탕
        //--- 펜 생성: 빨강색으로 펜을 설정
        HPEN hPen = CreatePen (PS_SOLID, 2, RGB(255, 0, 0));
        HPEN hOldPen = (HPEN) SelectObject(hdc, hPen);

        //--- 현재 ROP2 모드 적용
        SetROP2(hdc, currentROP2);

        //--- 선분 그리기
        MoveToEx(hdc, 200, 200, NULL);
        LineTo(hdc, 400, 400);

        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);

        EndPaint(hWnd, &ps);
        break;
    }
```

예 1) 래스터 연산 적용하여 선분 그리기(계속)

```
case WM_KEYDOWN:
    switch (wParam) {
        case 'A': currentROP2 = R2_COPYPEN; break;           //--- 펜 색 그대로
        case 'B': currentROP2 = R2_XORPEN; break;           //--- XOR
        case 'C': currentROP2 = R2_NOT; break;              //--- 반전
        case 'D': currentROP2 = R2_BLACK; break;            //--- 항상 검정
        case 'E': currentROP2 = R2_WHITE; break;            //--- 항상 흰색
        case 'F': currentROP2 = R2_NOTXORPEN; break;         //--- XOR 반전
        case 'G': currentROP2 = R2_MERGEPAEN; break;        //--- 펜과 배경 병합
        case 'H': currentROP2 = R2_MASKPEN; break;          //--- 펜과 배경 AND
        case 'I': currentROP2 = R2_NOTCOPYPEN; break;       //--- 펜 색 반전
    }
    InvalidateRect(hWnd, NULL, TRUE); // 다시 그리기 요청
    break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}
```

예 2) 고무줄 효과가 있는 직선그리기

LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

```
{
    HDC hDC;
    PAINTSTRUCT ps;
    static int startX, startY, oldX, oldY;
    static BOOL Drag;
    int endX, endY;

    switch (iMsg)
    {
        case WM_CREATE :
            startX = startY = 0;
            oldX = oldY = 0;
            Drag = FALSE;
            return 0 ;

            //--- 시작 좌표

        case WM_PAINT :
            hDC = BeginPaint (hWnd, &ps) ;
            MoveToEx (hDC, startX, startY, NULL);
            LineTo (hDC, oldX, oldY);
            EndPaint (hWnd, &ps) ;
            return 0 ;

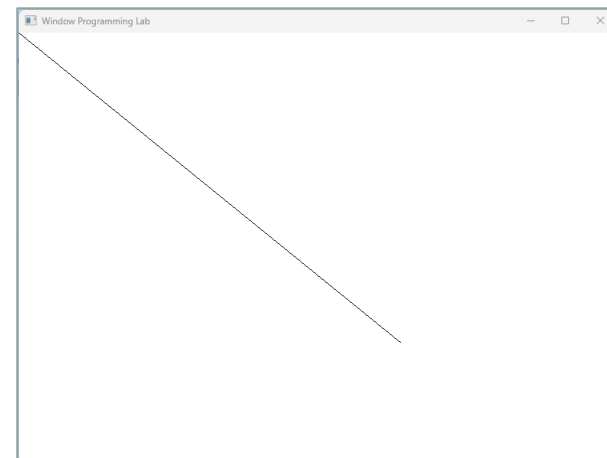
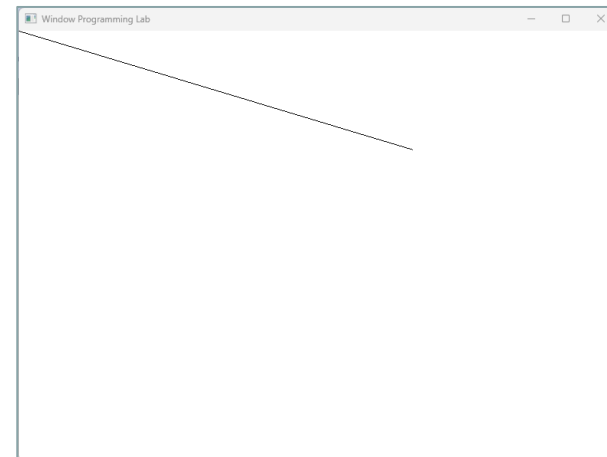
            //--- 이동하고 선으로 연결

        case WM_LBUTTONDOWN :
            Drag = TRUE;
            break;

            //--- 버튼을 누르면 드래그 동작 시작

        case WM_LBUTTONUP :
            Drag = FALSE;
            break;

            //--- 버튼을 놓으면 드래그 종료
    }
}
```



예 2) 고무줄 효과가 있는 직선그리기(계속)

```
case WM_MOUSEMOVE:
    hDC = GetDC (hWnd);
    if (Drag)
    {
        SetROP2 (hDC, R2_XORPEN);
        SelectObject (hDC, (HPEN)GetStockObject(WHITE_PEN));

        endX = LOWORD(lParam);
        endY = HIWORD(lParam);

        MoveToEx (hDC, startX, startY, NULL);
        LineTo (hDC, oldX, oldY);

        MoveToEx (hDC, startX, startY, NULL);
        LineTo (hDC, endX, endY);

        oldX = endX; oldY = endY;
    }
    ReleaseDC (hWnd, hDC);
    break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

}

return (DefWindowProc (hWnd, iMsg, wParam, lParam));
}
```

//--- 흰 바탕
//--- 펜의 XOR 연산:
//--- 두 비트가 서로 다를 때만 1을 반환 ($0^0 = 0$, $0^1=1$, $1^0=1$, $1^1=0$)
//--- 흰 펜
//--- 흰 바탕 XOR 흰 펜 = 검은색
//--- 검정 바탕 XOR 흰 펜 = 흰 색

//--- 지우기 : 검정 바탕 XOR 흰 펜 = 흰 선

//--- 그리기 : 흰 바탕 XOR 흰 펜 = 검은 선 → 화면의 결과 선

//--- 현 지점을 이전 지점으로 설정

3장 학습 내용

- 학습내용
 - 마우스 이벤트 다루기
 - 마우스 누를 때, 떼를 때, 움직일 때 발생 이벤트
 - 타이머 이벤트 다루기
 - 애니메이션 만들기
 - 래스터 연산
- 다음 시간에는
 - 리소스 사용하여 메뉴 만들기